

Introdução à Disciplina

Material de apoio — versão expandida e didática, para leitura antes ou depois da aula.

Prof. Daniel Plácido · Backend · SATC 2026.1

1. Bem-vindo(a) ao mundo do Backend

Se você nunca programou antes, ótimo — é exatamente daqui que todo mundo começa. Este material (e a disciplina como um todo) foi desenhado para partir do zero absoluto: cada termo técnico novo é explicado na primeira vez em que aparece, sem pressupor que você já conhece nada de programação.

Ao final do semestre, você será capaz de construir, sozinho(a), um sistema completo: uma **API** — explicamos esse termo em detalhe mais adiante — que recebe pedidos de um aplicativo ou site, aplica regras de negócio e guarda/consulta dados em um banco de dados. Chegaremos lá em etapas, uma camada por vez.

Uma regra importante desde já: se, em qualquer momento, um termo técnico usado em aula não fizer sentido, isso é sinal de que o professor precisa parar e explicar — não de que você está "atrasado". Pergunte sempre.

2. Quem vai te acompanhar nessa jornada

O professor responsável pela disciplina é o **Prof. Daniel Plácido**, que atua no mercado desenvolvendo e arquitetando sistemas backend usados por empresas reais no dia a dia — não se trata apenas de teoria de sala de aula. Ele também coordena iniciativas de tecnologia e estratégia dentro de uma empresa de software (ERP), o que o mantém próximo dos problemas reais que um desenvolvedor backend precisa resolver: performance, integrações entre sistemas, segurança da informação e organização de dados.

O objetivo da disciplina não é apenas ensinar a sintaxe de uma linguagem de programação — é ensinar a *pensar* como um desenvolvedor backend pensa quando precisa resolver um problema.

3. O que é, afinal, "programar"?

A analogia da receita de bolo

Uma receita de cozinha é uma sequência de passos que, seguida na ordem correta, sempre produz o mesmo resultado: "quebre 2 ovos", "misture com 1 xícara de farinha", "leve ao forno por 40 minutos". Programar é, essencialmente, escrever uma receita — chamada de **código** — que

o computador executa passo a passo, à risca, sem interpretar nem pular etapas.

A grande diferença entre um computador e uma pessoa seguindo uma receita é que o computador é extremamente literal. Se a receita disser "leve ao forno" mas ninguém disse antes "ligue o forno", o computador não vai adivinhar essa etapa faltante — ele simplesmente vai falhar ou produzir um resultado inesperado.

Traduzindo os termos que você vai ouvir bastante

Código	O texto com as instruções, escrito em uma linguagem que o computador entende (neste curso, usaremos Java).
Programa / sistema	O conjunto de instruções já pronto, rodando em um computador.
Executar / rodar	Quando dizemos que um programa está "rodando", significa que o computador está, nesse momento, seguindo as instruções escritas.
Bug	Um passo da receita (do código) que estava errado, incompleto ou mal pensado, e por isso gerou um resultado inesperado.

4. Frontend e Backend: a analogia do restaurante

Todo sistema com o qual você interage — um aplicativo de celular, um site — é dividido em duas grandes partes que trabalham em conjunto, de forma parecida com um restaurante.

O salão — Frontend

É a parte que o cliente vê e toca: a mesa, o cardápio, a decoração, o garçom. No mundo digital, o Frontend são as telas, os botões, as cores e os formulários — tudo aquilo que você vê e clica dentro de um aplicativo ou site. Exemplos: a tela de login do Instagram, o carrinho de compras de um site de e-commerce.

A cozinha — Backend

É a parte que o cliente **não** vê: o cozinheiro, o estoque de ingredientes, as receitas e as regras da casa ("esse prato só é servido depois das 18h"). No mundo digital, o Backend é o servidor que recebe o pedido do cliente, aplica as regras de negócio necessárias e guarda ou busca informações em um banco de dados. É essa camada — a que fica "por trás das cortinas" — que estudaremos ao longo de todo este semestre.

5. O que exatamente o Backend faz, passo a passo

1. Recebe pedidos	O backend escuta as solicitações que chegam do frontend: "quero fazer login", "quero ver meus pedidos anteriores".
--------------------------	--

2. Aplica regras de negócio	Decide o que é permitido ou não: "esse cupom de desconto só é válido para compras acima de R\$50", "este usuário não pode acessar dados de outro usuário".
3. Conversa com o banco de dados	Guarda informação de forma permanente (por exemplo, o cadastro de um novo cliente) e busca informação quando necessário (o histórico de pedidos de alguém).
4. Devolve uma resposta	Envia de volta ao frontend o resultado do processamento: os dados solicitados, uma confirmação de sucesso, ou uma mensagem de erro explicando exatamente o que deu errado.

6. Backend no seu dia a dia — exemplos concretos

iFood

Quando você faz um pedido, o backend verifica se o restaurante está aberto, calcula o valor da entrega com base na distância, reserva o pedido na fila de preparo da cozinha, e avisa o entregador disponível mais próximo.

Instagram

Quando você curte uma foto, o backend registra essa curtida, atualiza o contador exibido para todos, e decide — com base em diversos fatores — quem vai ver essa curtida aparecer no próprio feed.

Netflix

O backend sabe exatamente em que minuto do episódio você parou de assistir, sugere séries parecidas com base no seu histórico, e libera o acesso ao vídeo apenas para contas com assinatura ativa.

Aplicativo de banco

O backend confere seu saldo em tempo real, valida sua senha, aplica regras de segurança contra fraude, e só então autoriza (ou nega) uma transferência bancária.

7. Por que investir tempo aprendendo Backend?

Alta demanda	Toda empresa que possui um aplicativo, site ou sistema interno precisa de profissionais cuidando do que roda "por trás" desses produtos.
Oportunidades de trabalho remoto	Backend é uma das áreas de TI com mais vagas remotas, inclusive para empresas fora da sua cidade ou estado.
Base de conhecimento reaproveitável	As mesmas lógicas — API, banco de dados, regras de negócio — se repetem em praticamente todo sistema. Aprender uma vez permite aplicar o conhecimento em contextos muito diferentes.
Aprendizado contínuo valorizado	Um desenvolvedor backend raramente para de aprender — e isso é positivo: o mercado valoriza fortemente quem se mantém atualizado.

8. A Ementa: as três grandes etapas da disciplina

1ª Etapa — Ambiente de Desenvolvimento

Cobre os fundamentos da Web (como a internet e os navegadores funcionam), o protocolo HTTP (a "língua" usada entre navegador e servidor), e os fundamentos da linguagem Java: variáveis, orientação a objetos e lógica de programação. Encerra com a Avaliação N1.

2ª Etapa — API REST

Aborda como sistemas diferentes conversam entre si: o que é uma API, a arquitetura REST (um conjunto de regras muito usado no mercado), como persistir (salvar) dados em um banco de dados real, e como organizar os dados que entram e saem de uma API usando DTOs. Encerra com a Avaliação N2a (teórica) e o Projeto Intermediário N2b (prático).

3ª Etapa — Tópicos Avançados

Cobre integração entre sistemas e serviços externos, observabilidade e testes automatizados, o Model Context Protocol (MCP — um protocolo novo que permite que sistemas de Inteligência Artificial usem ferramentas de forma padronizada), autenticação e segurança, e infraestrutura. A disciplina se encerra com o desenvolvimento do Projeto Final, avaliado na N3.

9. Cronograma completo — 19 aulas

Aula 1	Introdução à Disciplina + Arquitetura da Web
Aula 2	Ambiente de Desenvolvimento
Aulas 3-4	Orientação a Objetos em Java (parte I e II)

Aula 5	Avaliação N1
Aula 6	Web Services + Protocolo HTTP/JSON
Aula 7	Padrões de Arquitetura + Arquitetura REST
Aula 8	Persistência de Dados
Aula 9	DTO e Mapeamento
Aula 10	Avaliação N2a
Aula 11	Projeto Intermediário N2b
Aula 12	Integração de Serviços
Aula 13	Observabilidade e Testes
Aula 14	Model Context Protocol (MCP)
Aula 15	Autenticação e Segurança
Aula 16	Infraestrutura
Aulas 17-18	Desenvolvimento do Projeto Final
Aula 19	Projeto Final — Avaliação N3

Datas específicas ainda a definir — o cronograma está sujeito a pequenos ajustes conforme o andamento da turma.

10. Como você vai ser avaliado(a)

N1 — Prova teórica	Cobre Arquitetura da Web, HTTP e Orientação a Objetos em Java. Formato prova escrita/individual.
N2a — Prova teórica	Cobre Web Services, arquitetura REST e Persistência de Dados.
N2b — Projeto prático	Você constrói uma API REST funcional, aplicando o que foi visto até aqui — é aqui que a teoria se transforma em prática de verdade.
N3 — Projeto Final	Sistema completo, integrando todos os tópicos avançados vistos na 3ª etapa da disciplina.

11. O LabTec: onde a prática acontece

O LabTec é o laboratório de tecnologia da SATC — o espaço físico onde as aulas práticas de programação acontecem. Diferente de estudar sozinho(a) em casa, no LabTec você programa e recebe feedback imediato do professor e dos colegas.

Computadores prontos para uso	Já vêm com as ferramentas básicas configuradas, então você não perde tempo de aula instalando programas do zero.
Regras de uso	Chegue no horário combinado, salve seu trabalho antes de sair (usaremos o Git/GitHub para isso — explicado a seguir) e mantenha o espaço organizado para a próxima turma que for usar a sala.

12. Onde encontrar todo o material: o repositório no GitHub

O que é o GitHub?

Pense no GitHub como um Google Drive feito especificamente para guardar código e arquivos de projetos de programação, com histórico de todas as versões já feitas. Um **repositório** (ou simplesmente "repo") é, nesse contexto, equivalente a uma "pasta" — um espaço onde ficam organizados todos os arquivos de um mesmo projeto ou disciplina.

Tudo que usarmos em aula — slides, materiais de apoio como este, e exemplos de código — vai ficar disponível no repositório da disciplina, sempre atualizado.

Como acessar, passo a passo

Passo 1	Acesse github.com/placidodaniel no navegador (não é necessário ter conta própria apenas para visualizar o conteúdo).
Passo 2	Localize a pasta da disciplina de Backend e, dentro dela, a pasta correspondente ao tópico da aula desejada.
Passo 3	Baixe os arquivos usando o botão "Download" ou "Raw", ou simplesmente leia o conteúdo diretamente pelo navegador.
Passo 4	Sempre que uma pasta for atualizada com material novo, ele estará disponível ali — não é necessário solicitar por e-mail.

13. Ferramentas que você vai ouvir bastante — glossário rápido

JDK	O "motor" que traduz e executa código Java no seu computador. Sem ele, o computador não entende instruções escritas em Java.
------------	--

IDE (ex.: IntelliJ)	Um editor de texto avançado, feito especialmente para escrever código, com recursos de ajuda automática (sugestões, correção de erros).
Git	Uma ferramenta que guarda o histórico de todas as versões do seu código, permitindo voltar atrás caso algo quebre ou dê errado.
GitHub	Um site que hospeda repositórios Git na nuvem (na internet), permitindo compartilhar e colaborar em projetos com outras pessoas.
Terminal	Uma tela de texto onde você digita comandos diretamente para o computador executar, em vez de clicar em ícones e menus.
API	Um "cardápio" de operações que um sistema oferece para que outros sistemas possam conversar com ele de forma organizada.
Banco de dados	Um sistema especializado em guardar informação de forma organizada, segura e permanente, mesmo que o computador seja reiniciado.
Postman	Um programa usado para testar APIs manualmente, simulando pedidos antes mesmo de existir um frontend pronto para isso.

14. Combinados de sala e comunicação

Dúvidas não têm hora errada	Pergunte durante a aula sempre que precisar — é bem provável que outros colegas tenham exatamente a mesma dúvida.
Sem julgamento por "pergunta básica"	Toda a turma está partindo do mesmo ponto de zero conhecimento prévio em programação, então não existe pergunta boba.
Presença e prática no LabTec	A prática em aula é o momento em que o conteúdo teórico realmente fixa — priorize sua presença nos dias de laboratório.
Canal oficial de comunicação	Avisos, materiais e alterações de cronograma serão comunicados pelo canal definido em aula, e sempre refletidos no repositório do GitHub.

15. Resumo da aula

- Backend é a "cozinha" de um sistema: regras de negócio, processamento e banco de dados.
- A disciplina está organizada em 3 etapas: Ambiente de Desenvolvimento → API REST → Tópicos Avançados.
- A avaliação combina provas teóricas (N1, N2a) com projetos práticos (N2b, N3).

- Todo o material da disciplina fica disponível e sempre atualizado no repositório do GitHub.

Próxima aula (Tópico 01): Introdução à Web e Arquitetura da Web — de onde vem a internet que usamos todos os dias.